

Lab 05: Container Orchestration

Operating Systems and Distributed Systems

This lab aims to introduce practical experience with utilising Kubernetes-based container orchestrators.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

Contents

Exercise 01: Install MiniKube	2
Pods	2
Deployments	2
Services	2
Exercise 02: Get to know Kubernetes	4
Exercise 03: Kubernetes Manifest	6
Kubernetes Manifest	6
Exercise 04: What am i doing?	8

Exercise 01: Install MiniKube

In this exercise, the student will install `minikube`, which is a small local Kubernetes installation used to both learn and develop for Kubernetes.

Visit minikubes website [<https://minikube.sigs.k8s.io/docs/start/>] to grab an appropriate Kubernetes installation for your architecture.

Information 1

The command `minikube start` can be used to set up the Kubernetes cluster. Conversely, the command `minikube delete --all` can stop and delete the cluster.

The following are brief introductions to the most essential Kubernetes objects. The introductions might be obsolete for this exercise, but in the upcoming exercise, they will become helpful.

Pods

Pods are the smallest unit of computing that can be created and managed in Kubernetes. A pod is a group of one or more containers with shared storage and network resources and a specification for how to run the containers. [Click for more information.](#)

Deployments

deploymentS are controllers describing which pods to run and how many instances of the pod to create by defining replicas. The deployment is defined in a YAML document, similarly to `docker compose`. [Click for more information.](#)

Services

services are used to manage the reachability of the pods from both inside and outside of the cluster. A pod inside a cluster has a cluster IP, but to make sure that pods can be created and destroyed dynamically, the cluster defines a single DNS name for a set of pods so you can reach the pod from other pods, even though the cluster IP might change. [Click for more information.](#)

NodePort

NodePorts exposes the Service on each Node's IP at a static port. Quite Literally. Read it a few times. "Exposes the service" means it will be open and available to connect to. But it is exposed on "each Node's IP", meaning that every node in the cluster will point to the same service on the specified port. It doesn't matter which node the service is running in. All nodes will point to the same service from the static port number. It is then possible to contact the service from outside the cluster using `<NodeIP>:<NodePort>`

Your task

1. Install and set up a Kubernetes cluster using `minikube`

Exercise 02: Get to know Kubernetes

In this exercise, the student will install `kubectl` on their machine. `kubectl` is the Kubernetes CLI enabling interaction with Kubernetes clusters. This will be the main tool used with Kubernetes, so getting familiar with this is very useful.

All `kubectl` commands follow a specific syntax to ease usage. The syntax is as follows:

```
$ kubectl <COMMAND> <RESOURCE TYPE> <NAME> <FLAGS>
```

Information 2

With `kubectl` namespaces other than the currently selected can be targeted by using the `-n` or `--namespace` flag. Additionally, `-A` or `--all-namespaces` can be used to target all namespaces.

Further information about `kubectl` can be found here: <https://kubernetes.io/docs/reference/kubectl/>

The following is a selection of useful commands that may be useful in the upcoming exercises.

`$ kubectl get <RESOURCES>` retrieves an overview of the resource in the Kubernetes cluster. Amongst many resources are nodes, pods, services and deployments.

A handy resource to request, especially for problem-solving, is `$ kubectl get events`. The reason can likely be pinpointed through the event log if unexpected behaviour is encountered, like a pod not working or a faulty deployment.

While the `get` commands are useful for getting an overview of the cluster, `describe` can be used to get in-depth information about a resource instance. `$ kubectl describe <RESOURCE> <NAME>`

Lastly, `$ kubectl port-forward <POD|deployment>/<NAME> <HOSTPORT>:>CONTAINERPORT>`, can be used to connect a port from within a pod to a local port. This is extremely useful when needing to debug a pod without exposing the pod and its contents to everyone.

Your task

1. Check out the following resources
 - (a) Nodes
 - (b) Namespaces
 - (c) Deployments
 - (d) Services
 - (e) Pods

2. In which namespaces are there currently resources present?

Exercise 03: Kubernetes Manifest

In this exercise, the student will create a Kubernetes Manifest (YAML-file) based on the compose file from Lab 02 – Exercise 05. If, for whatever reason, Lab 02 – Exercise 05 has not been finished, a compose file has been included in the resources.

Information 3

When a Manifest has been written, it can be applied unto a Kubernetes cluster, using

```
$ kubectl apply -f <PATH-TO-YAML>
```

Conversely, the Manifest can be removed from the Kubernetes cluster using

```
kubectl delete -f <PATH-TO-YAML>
```

Kubernetes Manifest

The manifest file(s) of Kubernetes is where one defines Kubernetes Objects such as deployments, services, namespaces, etc. Each type of object has a predetermined set of variables to be configured for Kubernetes to manage the configuration. By default, only a single object can be defined per Manifest. However, if desired, multiple Manifests can be merged into a single file, by separating them with ---.

```
1 apiVersion: apps/v1 # The apiVersion defining the the deployment
2 kind: Deployment # Defining which Kubernetes object to use
3 metadata: # Metadata for the Deployment
4   name: example-deployment # Name of Deployment
5 spec:
6   replicas: 3 # Number of replicas
7   selector:
8     matchLabels:
9       app: example-pod # This label serves as a reference to pod(s)
10  template: # Template for the Pod(s) to run
11    metadata: # Metadata for the Pod(s)
12      labels:
13        app: example-pod # Label matching the matchLabel
14    spec:
15      containers:
16        - name: hello-world # Name of the Pods
17          image: hello-world # Image running in the Pods
18          command: ["/bin/echo"] # Command to run
19          args: ["Message"] # Command arguments (and options)
20          env: # List of environment variables
```

```
21     - name: ENV_NAME
22       value: "Env value"
23     ports: # Port to expose
24     - containerPort: 8080
25 --- # This serves as a separation between the two objects
26 apiVersion: v1
27 kind: Service
28 metadata:
29   name: example-service
30 spec:
31   type: NodePort # Type of the service
32   ports:
33   - port: 80 # Cluster IP port to map to
34     targetPort: 8080 # Containerport to map from
35     nodePort: 32323 # Mapping to all nodes on port 32323
36   selector:
37     app: example-pod # This label serves as a reference to pod(s)
```

Exercise 04: What am i doing?

In this exercise, the students will discuss the topic of distributed systems and Kubernetes.

Your task

1. What is a control plane in Kubernetes, and what does it do?
2. What is a worker in Kubernetes, and what does it do?
3. What are the benefits of Kubernetes?
4. What are the disadvantages of Kubernetes?